

USP SECURE ENTRY SERVER®



UNITED SECURITY PROVIDERS

Documentation Series

HTTP Secure Proxy

Programmer's Guide

Version 4.27.0.5



United Security Providers AG
www.united-security-providers.ch
info@united-security-providers.ch

Headquarter	Stauffacherstrasse 65/15	CH-3014 Bern	Tel. +41 31 959 02 02
Baslerpark	Mürtschenstrasse 27	CH-8048 Zürich	Tel. +41 44 496 61 11



UNITED SECURITY PROVIDERS

Copyright © 2026 United Security Providers AG

This document is protected by copyright under the applicable laws and international treaties. No part of this document may be reproduced in any form and distributed to third parties by any means without prior written authorization of United Security Providers AG.

DOCUMENTATION IS PROVIDED "AS IS" AND ALL EXPRESSED OR IMPLIED REPRESENTATIONS AND WARRANTIES, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT, ARE DISCLAIMED TO THE EXTENT PERMISSIBLE UNDER THE APPLICABLE LAWS.



Contents

I	Overview	1
1	Introduction	3
1.1	Overview	3
1.2	Document Structure	3
1.3	Document Purpose	3
1.4	Target Audience	3
1.5	Terms	4
2	Architecture Overview	5
2.1	Overview	5
2.2	Login Service	6
2.3	Protection Levels	7
3	Interface Overview	8
3.1	Overview	8
3.2	HTTP Headers	8
3.2.1	Overview	8
3.2.2	HTTP Secure Proxy Header Filter	8
3.2.3	SSL-specific Headers	9
3.2.4	Sample Session	9
3.3	HTTP Cookies	10
3.3.1	Setting Cookies	10
4	Login Process Explained	11
4.1	Overview	11
II	Reference	15
5	Introduction	17
5.1	Overview	17
5.2	How to read the Reference	18



6	Client Interface	19
6.1	Overview	19
6.2	Specific Characteristics	19
6.3	Initial Request [1.1]	20
6.4	Initial Response [2]	20
6.5	Standard Request [7.1]	20
6.6	Standard Response [7.1]	21
6.7	Denied Access Response [3.4]	21
6.8	Successful Login Response (Authorization) [6.1]	22
7	Application Server Interface	23
7.1	Overview	23
7.2	Standard Request [7.1]	23
7.3	Standard Response [7.1]	25
7.4	Logout Response [8.1]	26
7.5	Re-authentication Response	27
8	Login Service Interface	28
8.1	Introduction	28
8.2	Authorization Request [4.1 4.2 4.3 4.4 4.5]	28
8.3	Successful Login Response (Authorization) [5.1]	30
9	HSP Internal Interface	32
10	Data Items	33
10.1	Item List	33
10.2	Session Info	37
10.3	Session Attributes	38
10.3.1	When To Create Session Attribute Headers	38
10.3.2	Session Attribute Header Syntax	38
10.3.2.1	Session Attribute Header Keys	39
10.3.2.2	Supported Encodings	39
10.3.2.3	Supported values for the "vtype" key	40
10.3.3	Usage Types	40
10.3.3.1	HTTP Header Propagation (usage="prop-header")	40
10.3.3.2	Gateway Parameter (usage="gw-param")	40
10.3.3.3	AAI Parameter (usage="aai-param")	41
10.3.3.4	Application Authorization Identifier (usage="ac-app-az")	41
10.3.4	Examples	41



10.3.4.1 Basic	41
10.3.4.2 Gateway Parameter	41
10.3.4.3 Exact Match	41
10.3.4.4 vtype	42
10.4 Airlock Backend API Emulator	42
10.5 Airlock Enviroment Cookies]	42
A Document References	43
B Glossary	44



Part I

Overview



The following part contains a general overview to the HSP developer. It shows the overall architecture and defines terms that are used later in the reference part.



Chapter 1

Introduction

1.1 Overview

This document contains detailed technical descriptions of all interfaces that web application developers need to know and understand in order to develop USP Secure Entry Server®-enabled web applications and login services.

1.2 Document Structure

This document contains the following parts:

The current chapter, "[1 Introduction](#)", gives you an overview of the document, explains its purpose and what target audience it has. It also defines terms used in this document.

Chapter

The third part provides a simple yet complete example of a servlet structure. Source code fragments are shown with gray background, to distinguish them from surrounding text.

Knowledge of Java programming and HTTP/HTTPS is necessary to understanding this document.

1.3 Document Purpose

This document is intended as a guide for web application and login service developers, primarily focussing on the Java programming language and developer kits.

1.4 Target Audience

This document is intended for the following professionals:

- Login Service developers, especially Java developers, to be able to provide custom authentication mechanisms
- All other developers, to find the interface definitions between the HSP and a login service implementation
- Application integrators, to understand the interface definitions between the HSP and the protected web applications
- Software Architects, to understand the interfaces between all involved components (HSP, login service, protected web application)



1.5 Terms

Table 1.1: Terms and Definitions

Term	Definitions
SCDID	Secure Data Client ID. This ID identifies each Client (Web Browser Instances) and references the Client Session which is used by the "Session/Credential Management" to store credentials represented by cookies.
SES	USP Secure Entry Server®, product name for all reverse proxy components: HSP, SLS, SRM, SZGW, etc.
HSP	HTTP Secure Proxy, HTTP-based reverse proxy
SLS	Secure Login Service, name for the login service web application used in this document
HTTP Redirect	Special HTTP Response which causes HTTP Clients to immediately access another URL.
Session Credential	Authorizes access to a specific path (URL) and Access Area. May include additional data such as the User Id.
Client Session	The "Client Session" session is automatically generated for each new client. This session is not visible to the client and does not require a login and not provide or grant access to any data. The Client Session is used to transparently store Credentials sent by Application Servers.
Client Data	Contains the Client Session, See "Client Session"
CDS	Client Data Store, used to store client data, e.g. cookies in a store located within a single HTTP Secure Proxy
Authorization	A HTTP Header sent from a Login Service which authorizes the Client for a specific Access Area.



Chapter 2

Architecture Overview

This chapter gives you a short introduction to the overall architecture of the HTTP Secure Proxy.

2.1 Overview

The HTTP Secure Proxy is a HTTP reverse proxy that protects web applications by providing externalization of authentication to a specific login service, i.e. the HTTP Secure Proxy does not perform authentication itself but intercepts each client request and validates its access rights.

One of the advantage of the HTTP Secure Proxy is its focus on use of non-intrusive, standardized methods for the interfaces between all involved components, which are shown in the following image:

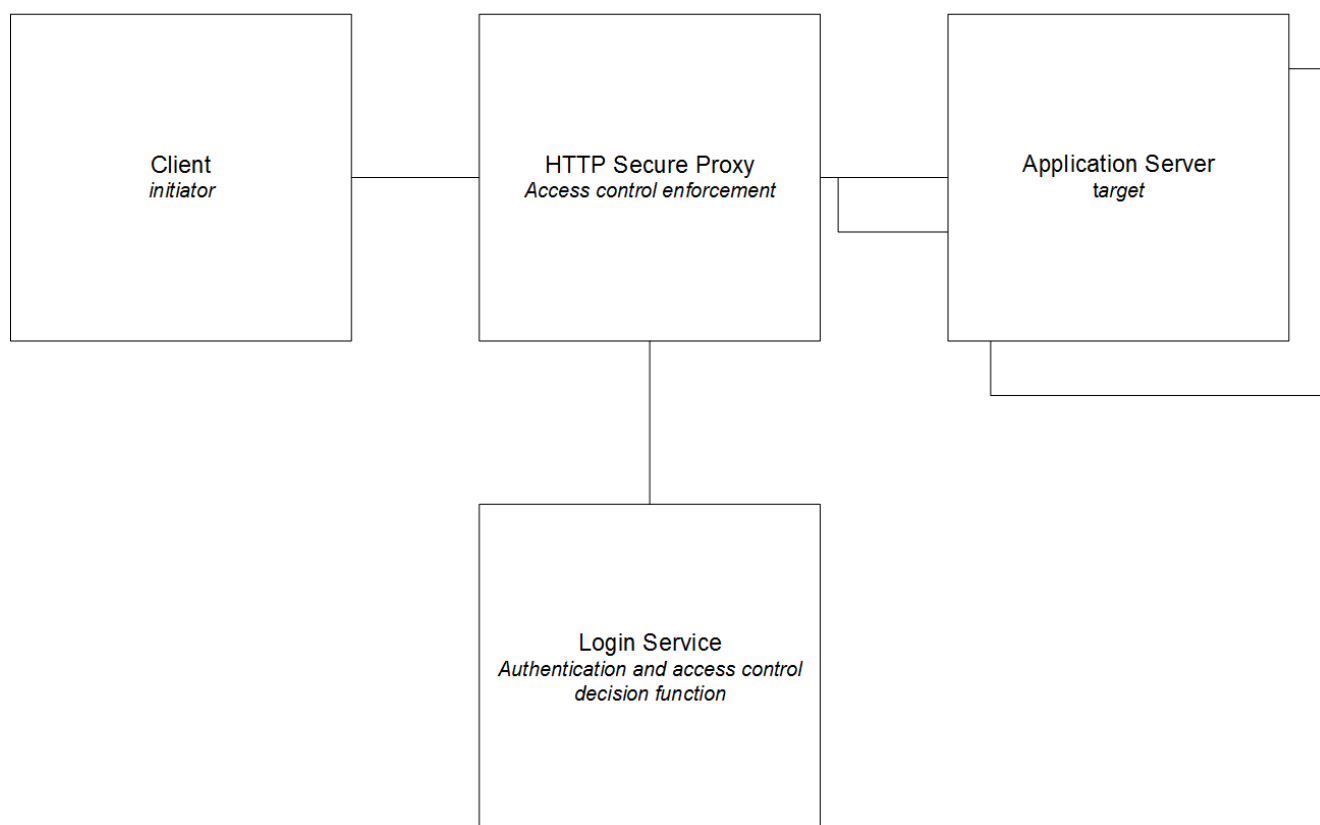


Figure 2.1: Architecture overview

The client, also called "initiator", is typically a standard Web browser. A user types in a specific URL protected by a HTTP Secure Proxy, e.g. <https://entry.mycompany.com/app/portal>.

The HTTP Secure Proxy, called "access control enforcement" above, redirects the request to the Login Service, called "authentication and access control decision function" above.

The login service is responsible for the whole authentication process. When the user has successfully authenticated herself, it redirects the user request back to the original web application URL. It does this by using the interface between the Login Service and the HTTP Secure Proxy, which is solely based on HTTP mechanisms.

At last, the HTTP Secure Proxy allows the request initiated by the client to travel to the web application, called "target" in the image above.

To resume, the HTTP Secure Proxy forwards request and enforces access control based on locations.

2.2 Login Service

While the HTTP Secure Proxy implements access enforcement functions by redirecting the client to the Login Service, the Login Service implements the access control mechanisms by authenticating the client.

The Login Service therefore implements the login process including all necessary dialogs with the client.



2.3 Protection Levels

Each location in the HTTP Secure Proxy can be defined to carry one of the access levels defined above by setting a specific directive. Please refer to the Administrator's Guide of the HTTP Secure Proxy for more details about configuration directives.

The following protection levels are possible:

- **Public** : A location defined "public" is accessible by all visitors. The HTTP Secure Proxy does not perform access control on it, but simply verifies the HTTP or HTTPS headers to block potential attacks on known security holes in web or application servers.
- **Member** : For locations defined "member" level, visitors must typically identify themselves with a so-called "weak" authentication method, e.g. user name and password in order to be able to access the web application behind the location. As described above, the HTTP Secure Proxy checks each request and, if not authenticated yet, redirects the user to the login service. The login service then typically provides a login form with the appropriate authentication mechanism.
- **Customer** : The third and most secure level is the "customer" level. A simple user name/password check is insufficient for visitors to gain access. We recommend the use of security certificates or RSA SecurID authentication. It is again the responsibility of the Login Service to grant access to different levels appropriately.



Chapter 3

Interface Overview

The HTTP Secure Proxy uses different interfaces to interact with the Login Service, the application servers and the client. This chapter defines each interface and gives you details and examples about it.

3.1 Overview

Generally, the interfaces between all involved components are based on the HTTP standard. Headers and cookies are used to signal state changes, failures and successes, and to deliver information between the components. The following interfaces

Client interface

the interface between a client, e.g. a Web browser and the HTTP Secure Proxy

Application server interface

the interface between a protected Web application and the HTTP Secure Proxy

Login Service interface

the interface between the Login Service and the HTTP Secure Proxy

An implicit interface exists between the protected Web application and the Login Service as the Login Service delivers authentication, or identity, information to the protected Web application.

3.2 HTTP Headers

3.2.1 Overview

According to [\[RFCHTTP1.1\]](#) supports transmission of special header data that is not directly visible to the Web browser client. Applications can use this extra information for content classification. The HTTP Secure Proxy and the Login Service use HTTP headers for storing control information. Applications and the HTTP Secure Proxy communicate directly via the header data field, while cookies are used between the Web browser client and the HTTP Secure Proxy.

3.2.2 HTTP Secure Proxy Header Filter

The HTTP Secure Proxy filters header data fields, passing only its own fields and external fields that have been explicitly enabled by the administrator. If your Web application needs to receive specific HTTP headers, you need to provide this



information to the HTTP Secure Proxy administrator. The person in charge there is then able to configure header names in your location definition as not being blocked.

Regular browsers are generally incapable of handling custom headers, and they only generate headers envisaged by the standard. However, other client types, e.g. Java applications or Web Services clients may set custom headers with each request, hence you must be aware of the HTTP header filter feature of the HTTP Secure Proxy.

3.2.3 SSL-specific Headers

SSL information like Fingerprint, Certificate, SerialNumber, etc. is an important case in point when you want to verify a security certificate (exchanged between the HTTP Secure Proxy and the Login Service web application). Available headers and their potential applications are described in [\[HTTPADMIN\]](#).

3.2.4 Sample Session

The following scenario provides an example of what happens mainly between the HTTP Secure Proxy, the Login Service and the protected Web application. We use Java and the Servlet API standard as the environment for the source code we provide here.

A client enters an URL in the Web browser. The HTTP Secure Proxy intercepts this request and redirects the Web browser to the Login Service. The HTTP Secure Proxy provides the Login Service with different information, e.g. the requested page. Therefore, the Login Service stores this value by reading it from a specific header sent to it:

```
String requestedPage =  
    HttpServletRequest.getHeader("RequestedPage");
```

The Login Service presents a login form to the user. After successful login, the Login Service sets two headers and redirects the Web browser client to the original URL stored before:

```
HttpServletResponse.setHeader("AccessArea", "Member");  
HttpServletResponse.setHeader("LoginUserData",  
    "id=bgo;option1=10;option2=blue");  
HttpServletResponse.sendRedirect(requestedPage);
```

By setting the header "AccessArea" to "Member", the Login Services signals a successful authentication back to the HTTP Secure Proxy.

A second header, "LoginUserData", set by the Login Service contains the identity information of the authenticated user. This header is carried forward by the HTTP Secure Proxy with each request that the protected Web application receives.

After this redirect, the HTTP Secure Proxy now receives the redirect to the original URL. The user is now authorized for this location, the HTTP Secure Proxy passes the request to the application server host and port defined in the location, adding the login credentials (provided by the Login Service in the "LoginUserData" header) to each request in the header "LoginCredentials". Therefore, a very simple Web application finds out about the identity of the authenticated user by issuing the following command:

```
if (no session available yet) {  
    String loginCredentials =  
        HttpServletRequest.getHeader("LoginCredentials");  
    // verify credentials, create session, etc.  
    String id = retrieveId(loginCredentials);  
    ...  
    // create session  
    HttpServletRequest.getSession(true);  
    // set the user in the application or on  
    // container level  
    ...  
}
```



It is, of course, better to either use the Java Servlet Filter approach or JAAS for a cleaner, more modular solution. This is, however, not in the scope of this document.

3.3 HTTP Cookies

3.3.1 Setting Cookies

You are able to set, update or remove cookies in the Login Service and in the protected Web applications.

The HTTP Secure Proxy stores client-related cookies in the Client Data Store (CDS). The client only gets a reference to its data items in the CDS. All cookies stored in the Client Data Store are lost after a logout of the user.

It is possible to configure cookies as being "transparent". Such cookies are not stored in the Client Data Store, they are passed through to the client. This feature can be used to persist cookies in the client browser, e.g. for storing default language configuration, specific, not security-relevant user account information. If such cookies are not made "transparent", stored information is lost at each logout.



Chapter 4

Login Process Explained

4.1 Overview

The following sequence diagram shows all interaction scenarios that appear in the context of the HTTP Secure Proxy. It provides an overview of all possible states of the architecture. The follow-up table gives more details about each state and its involved data flows. Please note that the next chapter then provides detailed information about each data item that are just mentioned here. We provide links to the provided sections wherever possible.

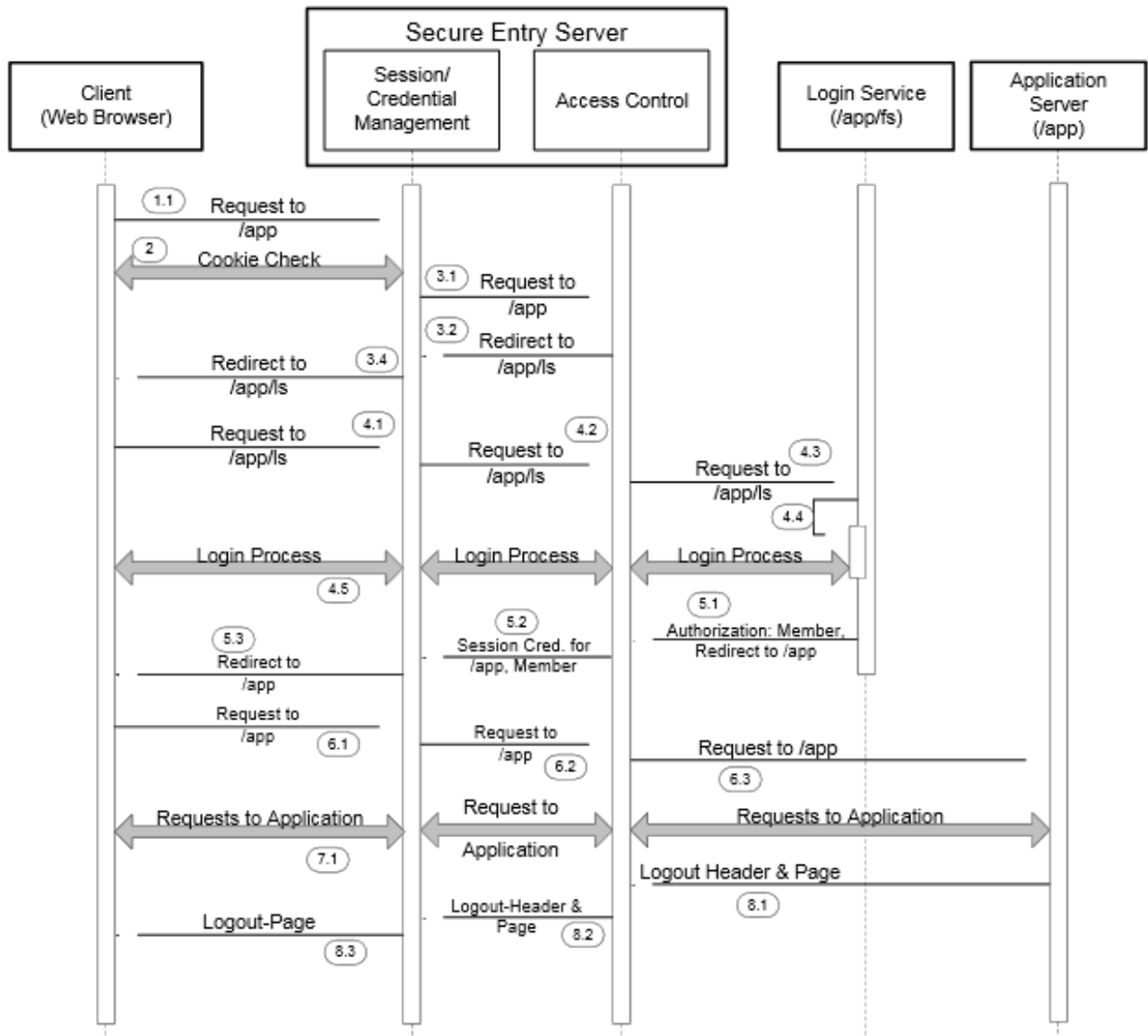


Figure 4.1: Login process sequence diagram

The table below, presents more details about the process shown above. The text typeset in `fixed width font` shows a simplified HTTP Request.



Table 4.1: Details about the login process sequence diagram

Id	Type	Description
1.1	Message	The initial client request, targeting an application (/app) GET /app HTTP/1.0
	Action	Since the client has not sent a valid SCDID, a cookie check will be initiated. (If cookie-check is disabled, step <2> is skipped.)
2	Messages	The cookie-check method uses HTTP redirects in order to make sure the client has accepted the SCDID cookie.
	Action	Error page or redirect to the URL specified in <1.1>.
3.1	Message	The original request from <1.1> is passed to the HTTP Secure Proxy access control module. GET /app HTTP/1.1
	Action	Client has no credential, a HTTP redirect message targeting the Login Service is sent back to the client.
3.4	Message	HTTP Redirect targeting the Login Service: HTTP/1.1 302 Moved Temporarily Location: /app/ls?RequestedPage=%2fapp
	Result	The client is redirected to the Login Service.
4.1	Messages	HTTP Messages for a form-based authentication of the client.
4.2		
4.3		
4.4		
4.5		
5.1	Message	HTTP redirect containing an authorization for a specific access area and URL. The HTTP redirect is targeting the original URL specified in <1.1>. HTTP/1.1 302 Moved Temporarily Location: /app AccessArea: Member
	Action	The HTTP Secure Proxy removes the authorization from the request and generates a valid credential for the authorization given by the Login Service.
5.2	Message	Forwarded HTTP redirect from <5.1>. Contains the HTTP Secure Proxy session credential which replaces the authorization from <5.1>. HTTP/1.1 302 Moved Temporarily Location: /app Set-Cookie: SessionCredentials=ab324cdfds...
	Action	The HTTP Secure Proxy session store module removes all cookies set by applications from the HTTP responses and restores them in later requests.
5.3	Message	HTTP redirect from <5.1> HTTP/1.1 302 Moved Temporarily Location: /app
	Action	Redirect to URL from <1.1>
6.1	Message	Original request from <1.1>, including SCDID GET /app HTTP/1.0 Cookie: SCDID="h9q86GS..."
	Action	The HTTP Secure Proxy replaces SCDID by the session credentials
6.2	Message	Request from <6.1>, with HTTP Secure Proxy session credentials. GET /app HTTP/1.0 Cookie: SessionCredentials="HYRTF..."
	Action	The HTTP Secure Proxy access control module verifies the session credentials and extracts the login user data.



Table 4.1: (continued)

Id	Type	Description
6.3	Message	Request from <6.2>, with Login User Data and session credentials. GET /app HTTP/1.0 LoginUserData: name=xyz, id=1234
	Action	After verifying the LoginUserData header and an optional login ticket (not shown here), the Web application creates a new session.
7.1	Messages	Any cookies, e.g. the session ID sent by the application server will be stored in the Entry Server's CDS.
	Actions	Inactive time-out handling,
8.1	Message	HTTP response, containing an "Invalidate" or "InvalidateAll" message and a HTML page confirming the logout. HTTP/1.0 200 OK AccessArea: Invalidate <HTML> (Logout Page) </HTML>
	Action	The invalidate message is internally forwarded to the session/credential management and removed from the HTTP response
8.2	Message	Response from <8.1> without the invalidate message.
	Action	On "Invalidate": All credentials stored within the application's path (/app) are removed from the HTTP Secure Proxy's session store. On "InvalidateAll": All data accosted with the client is deleted from the HTTP Secure Proxy's session store regardless of it's path.
8.3	Message	Response from <8.1> without the invalidate message.
	Action	-



Part II

Reference



The following part contains the detailed reference of all involved interfaces.



Chapter 5

Introduction

5.1 Overview

This chapter contains detailed specification of the HTTP Secure Proxy interfaces. For a quick overview, you may prefer to start with the sequence diagram at chapter ["Login Process Explained"](#). The following image repeats the main components and labels the different interfaces:

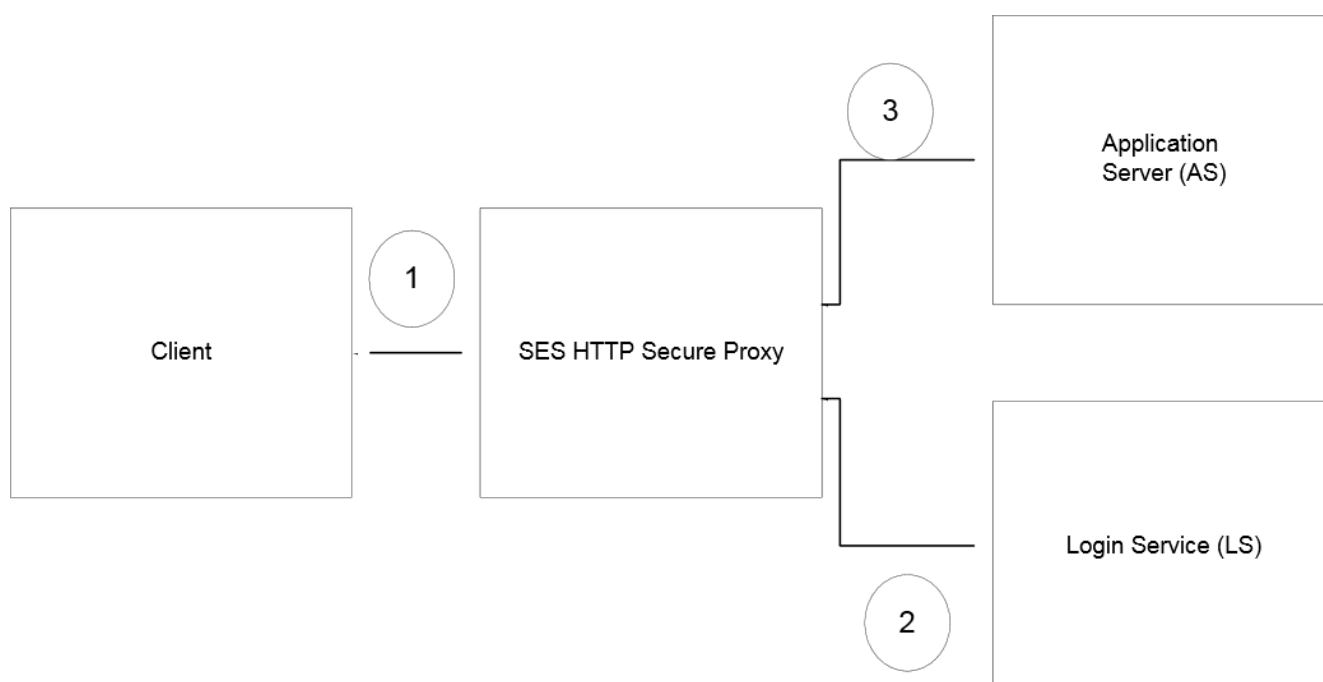


Figure 5.1: Interface Overview

1. The interface between the client and the HTTP Secure Proxy is defined in chapter ["6 Client Interface"](#).
2. The interface between the Application Server (i.e. the protected Web application) and the HTTP Secure Proxy is defined in ["7 Application Server Interface"](#)
3. The interface between the Login Service and the HTTP Secure Proxy is defined in ["8 Login Service Interface"](#)



Chapter "9 HSP Internal Interface" defines internal-only interfaces.

The following chapters "11 Data Items", "11.2 Session Info" and "11.3 Session Attributes" contain the definitions of all data items used in the different interfaces.

HTTP headers and cookies are used to exchange information between the different components. The generic term 'data item' is used in the following reference.

All syntax definitions base on the syntax rules and definitions specified in [RFCHTTP1.1], section "Basic Rules".

5.2 How to read the Reference

Keywords are written in italic style (*like this* or *like this*). Make sure you follow all keywords to get all the provided information. If you are reading this document using the Acrobat PDF reader, you can click on keywords to navigate quickly through the document.

References to the sequence diagram from chapter "4 Login Process Explained" are marked as **[id]** where *id* references the identifier in Table 2.

Furthermore, the following acronyms are used in the reference:

HSP	HTTP Secure Proxy
ESR	External Session Representation, see [HTTADMIN] for details.
LS	Login Service
AS	Application Server

Additionally, the following terms that are used in the reference:

Item

Information in the HSP system of a specific *item type*. Transported in HTTP requests or responses using a specific *encoding* and *container*.

Item type

Specifies the class of the *item*, see chapter "11 Data Items".

Container

Specifies where the *item* is transported, e.g. in a HTTP header or HTTP cookie or in the URI.

Encoding

The encoding, e.g. plain, URL or Base64.

Name

Name of the container, e.g. name of the HTTP header.

The following interface definitions assume that the listed headers are not blocked by HSP configuration. All examples assume default configuration.



Chapter 6

Client Interface

6.1 Overview

This section is for the client developer who intends to integrate HTTP clients with the HSP.

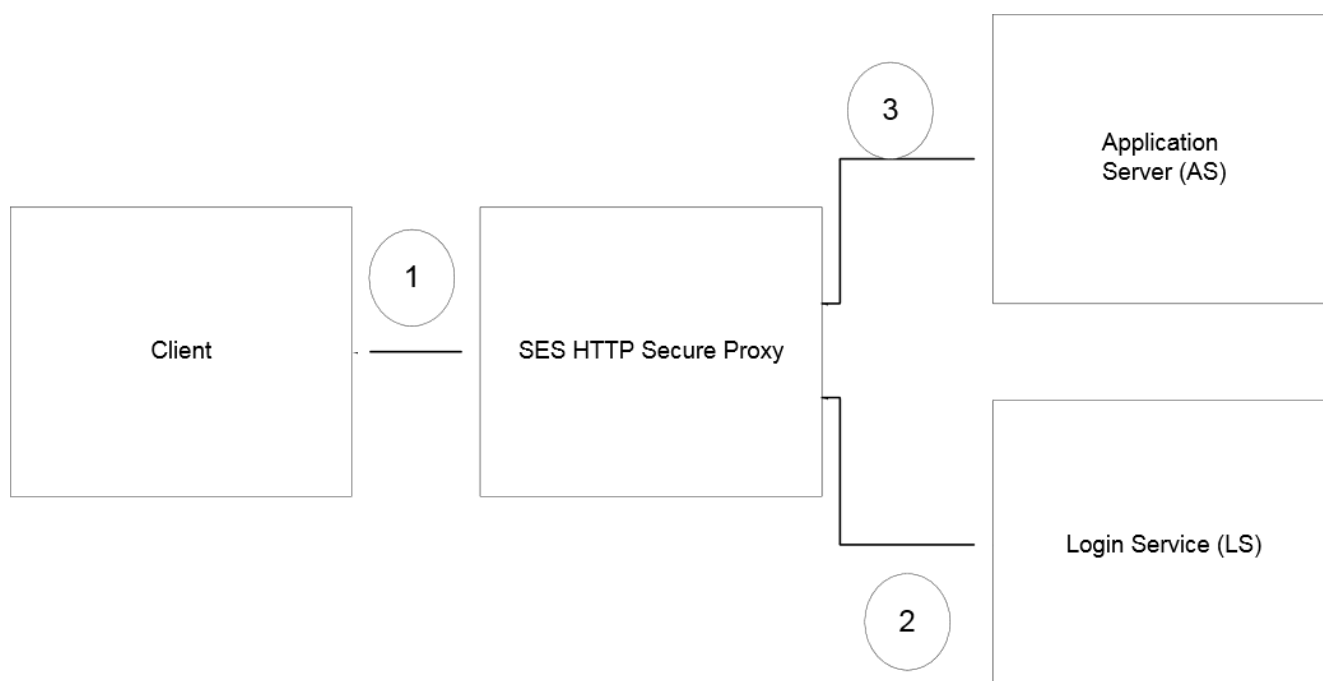


Figure 6.1: Client interface

6.2 Specific Characteristics

The following bullet list contains general remarks about the client interface:

- Cookies are not passed back and forth through the HSP unless they are configured as "transparent" in the HSP configuration. See the directive `SE_IntCookie_PersistentCookies` in [\[HTTPADMIN\]](#).



- Requests using the HTTP transfer encoding type "chunked" (chunked POST) are not supported and will be blocked. POST bodies must use a "Content-Length" header.
- The HTTP "Server" Header sent by application servers is overwritten by the HSP. The new header content can be configured.
- The behavior of the HSP can be changed by programmatically setting a specific value to the HTTP "User-Agent" header in the client application. See the documentation about the directives *SE_ExtCookie_CookieCheck* and *SE_ExtSslSid_Supported* in [\[HTTPADMIN\]](#) for further information.

6.3 Initial Request [1.1]

The first request to the HSP does not contain any HSP-specific data items. If the all-first request uses the HTTP POST method, the POST data is stored in the post data store of the HSP. After successful cookie check and, if necessary, successful login, the POST data is re-issued to the target URL.

6.4 Initial Response [2]

The initial response uses a HTTP "302 Moved Temporarily" message to set the SCDID.

Table 6.1: [Client Interface - Initial Response \[2\]](#) - Information Items

Item	Name, Description, Conditions, Sample
SCDID	Item Type: SCDID Encoding: If ESR "Cookie" is used. The HSP sets the SCDID as secure Cookie. If ESR "URL" is used, the HTTP "Location" header contains the SCDID as the first directory of the URI Path. Cookie Name: The cookie name starts always with "SCDID", a configurable and dynamic part is added, default name is "SCIDID_S" Sample HTTP "Set-Cookie" Header: <code>Set-Cookie: SCDID_S=SCDID; path=/; Secure</code> Sample HTTP "Location" Header: <code>Location: https://entry.foo.com/SCDIDSCDID/app/index.html</code>

6.5 Standard Request [7.1]

Standard requests are used after the ESR has been properly inialized.

Table 6.2: [Client Interface - Standard Request \[7.1\]](#) - Information Items

Items	Name, Description, Conditions, Samples
SCDID	Item Type: SCDID Encoding: If ESR "Cookie" is used. The SCDID is sent as secure Cookie. If ESR "URL" is used, the SCDID is sent as part of the URI Path. Sample "Cookie" Header: Cookie: SCDID_S= SCDID Sample HTTP Request Line: GET /SCDID SCDID /app/index.html HTTP/1.1

6.6 Standard Response [\[7.1\]](#)

Standard responses are used after the ESR has been properly initialized.

Table 6.3: [Client Interface - Standard Response \[7.1\]](#) - Information Items

Items	Name, Description, Conditions, Samples
SCDID <optional>	Item Type: SCDID If ESR "URL" is used, the SCDID is sent in Location headers. Sample HTTP "Location" Header: Location: https://entry.foo.com/SCDID SCDID /app/index.html
Session Timeout <optional>	Item Type: Session Timeout Encoding: Plain Container: Cookie Name: Defined with AC_HspCredentialTimeoutCookie directive Sample Cookie: <Name>=<since 00:00:00 UTC, January 1, 1970 in seconds>; path=<path>; domain=<domain>

6.7 Denied Access Response [\[3.4\]](#)

The "[Denied Access Response \[3.4\]](#)" is based on the "Standard Response"

On denied access, HSP sends "302 Moved Temporarily" which redirects the client to the login service. This response contains the following items:

Table 6.4: [Client Interface - Denied Access Response \[3.4\]](#) - Information Items

Item	Name, Description, Conditions, Samples
Requested Page <optional>	Item Type: Requested Page Encoding: URL encoded Container: URL parameter in the "Location" Header. Name: "RequestedPage" Sample: RequestedPage=<URL encoded Requested Page >
Login Request Key <optional>	Item Type: HSP Login Request Key Encoding: URL encoded Container: URL parameter in the "Location" Header. Available only in case of a HTTP POST method. Name: "HSPClientDataKey" Sample: HSPClientDataKey=<URL encoded HSP Login Request Key >
Post Data Storage Info <optional>	Item Type: Post Data Storage Info Encoding: URL encoded Container: URL parameter in the "Location" Header. Available only in case of a HTTP POST method. Name "PostDataStorageInfo" Sample: PostDataStorageInfo=<URL encoded Post Data Storage Info >
Future Improvement?	

6.8 Successful Login Response (Authorization) [6.1]

The authorization response authorizes the client for a specific access area and URI path. The access area is specified by the Login Service, whereas the URI path is configured in the HTTP Secure Proxy.

Usually, this response is sent using a HTTP "302 Moved Temporary" status which redirects the client back to the application.

Table 6.5: [Client Interface - Successful Login Response \(Authorization\) \[6.1\]](#) - Information Items

Item	Name, Description, Conditions, Sample
Login Request Key <optional>	Specifies the login request key. This value must be specified in order to support POST request playback. Item Type: HSP Login Request Key Encoding: URL encoded Container: URL parameter in the "Location" Header. Name: "HSPClientDataKey" Sample: HSPClientDataKey=<URL encoded HSP Login Request Key >



Chapter 7

Application Server Interface

7.1 Overview

The application server interface defines the communication means between the application server and the HTTP Secure Proxy.

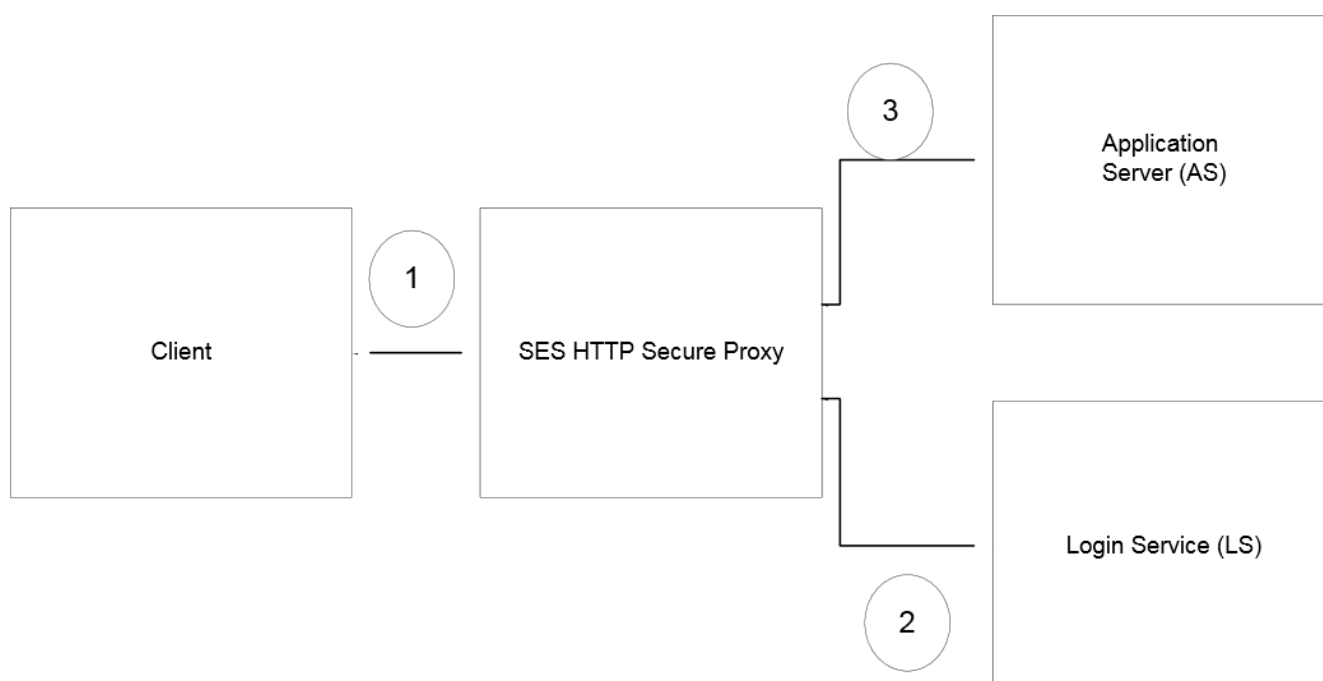


Figure 7.1: Application server interface

HSP generates information using HTTP headers which can optionally be evaluated by applications. Further, applications can optionally control the SES behavior by setting HTTP headers in their response.

7.2 Standard Request [\[7.1\]](#)

A request to an application server contains the following information items:

Table 7.1: [Application Server Interface - Standard Request \[7.1\]](#) - Information Items

Items	Name, Description, Conditions, Sample
Client Correlator	Item Type: Client Correlator Encoding: Plain Container: HTTP Header Name: ClientCorrelator Sample Header: ClientCorrelator: <Client Correlator>
Request Correlator	Item Type: Request Correlator Container: HTTP Header Encoding: Plain Name: RequestCorrelator Sample Header: RequestCorrelator: <Request Correlator>
ConnectionCorrelator	Item Type: Connection Correlator Container: HTTP Header Encoding: Plain Name: ConnectionCorrelator Sample Header: ConnectionCorrelator: <Connection Id>
Application ID	Item Type: Application Id Encoding: Plain Container: HTTP Header Name: ApplicationID Sample Header: ApplicationID: <Application Id>
Client Address	Item Type: Client Address Encoding: Plain Container: HTTP Header Name: HSP_CLIENT_ADDR Sample Header: HSP_CLIENT_ADDR: <Client Address>
Login Credentials	Item Type: Login Credentials Encoding: Plain Container: HTTP Header Name: LoginCredentials (default, depends on configuration)
SSL Headers (optional, multiple)	Item Type: SSL Headers Encoding: Plain Container: Multiple HTTP Headers Name: Multiple Names
HSP Listener Host	Item Type: HSP Listener Host Encoding: Plain Container: HTTP Header Name: HSP_HTTPS_HOST
Listener URI	Item Type: HSP Listener URI Encoding: Plan Container: HTTP Header Name: Hsp-ListenerUri



7.3 Standard Response [7.1]

The application server response can contain the following items:

Table 7.2: [Application Server Interface - Standard Response \[7.1\]](#) - Information Items

Items	Name, Description, Conditions, Sample
Response Filter Deactivation <optional>	Item Type: Response Filter Deactivator Encoding: Plain Container: HTTP Header Name: "HSP_NoResponseFiltering" Sample Header: HSP_NoResponseFiltering:
Logout Command <optional>	Item Type: Logout Command Encoding: Plain Container: HTTP Header Name: "AccessArea" Sample Header: AccessArea: < Logout Command >
Server load information <optional>	Item Type: Load Information Encoding: Plain Container: HTTP Header Name: "HSP_SERVER_LOAD_INFO" Sample Header: HSP_SERVER_LOAD_INFO: < Application Server Load >
URL Encoding <optional>	Item Type: URL Encoding Encoding: Plain Container: HTTP Header Name: "HspUE_EncPath", 'HspUE_EncQuery\' and 'HspUE_EncQuery_INPUT' Sample Header: HspUE_EncPath: on HspUE_EncQuery: session uid credit
Access Control Events <optional>	Item Type: Access Control Events Encoding: Plain Container: HTTP Header Name: HSP_C_AC Values: AC(10):login_member AC(11):login_customer AC(12):invalidate AC(13):invalid AC(14):insufficient AC(15):startpage AC(16):az AC(17):az soap Sample Header: HSP_C_AC: AC(10):login_member



Table 7.2: (continued)

Items	Name, Description, Conditions, Sample
Request Filter Events	Item Type: Request Filter Events Encoding: Plain Container: HTTP Header Name: HSP_C_RF Values: RF(30):RQ_violation RF(31):RS_violation RF(32):UE_violation RF(33):FV_violation RF(34):CE_violation RF(35):SF_violation RF(36):VR_violation Sample Header: HSP_C_RF: RF(30):RQ_violation;RF(34)CE_violation
Session Events	Item Type: Session Events Encoding: Plain Container: HTTP Header Name: HSP_C_SE Values: SE(50):new SE(51):invalidate_all SE(52):invalidate SE(53):LK_violation SE(54):dos SE(55):transfer SE(56):implicit_reauth Sample Header: HSP_C_SE: SE(50):new
Gateway Events	Item Type: Gateway Events Encoding: Plain Container: HTTP Header Name: HSP_C_GW Values: GW(74):failover Sample Header: HSP_C_SE: GW(74):failover

7.4 Logout Response [\[8.1\]](#)

This response logs the client out of a specified set of applications by deleting a set of cookies. Usually, this response is sent with a HTTP"302 Moved Temporarily" status which redirects the client to a public page which confirms the log out operation.

Table 7.3: [Application Server Interface - Logout Response \[8.1\]](#) - Information Items

Items	Name, Description, Conditions, Sample
Logout Command <optional>	Item Type: Logout Command Encoding: Plain Container: HTTP Header Name: "AccessArea" Sample Header: AccessArea: <Logout Command>

7.5 Re-authentication Response

This response initiates the re-authentication process. During re-authentication, access to protected locations is denied, unless the user has successfully identified against the login service. In contrast to the logout response, client session (i.e. cookies etc.) will not be deleted and is again available after re-authentication.

Table 7.4: [Application Server Interface - Re-authentication Response](#) - Information Items

Items	Name, Description, Conditions, Sample
Re-authentication Command <optional>	Item Type: Re-authentication Command Encoding: Plain Container: HTTP Header Name: "AccessArea" Sample Header: AccessArea: <Re-authentication Command>



Chapter 8

Login Service Interface

8.1 Introduction

The HSP Login Service interface bases on "[Application Server Interface](#)" and extends it. All information sent by HSP to Applications is available to the login server.

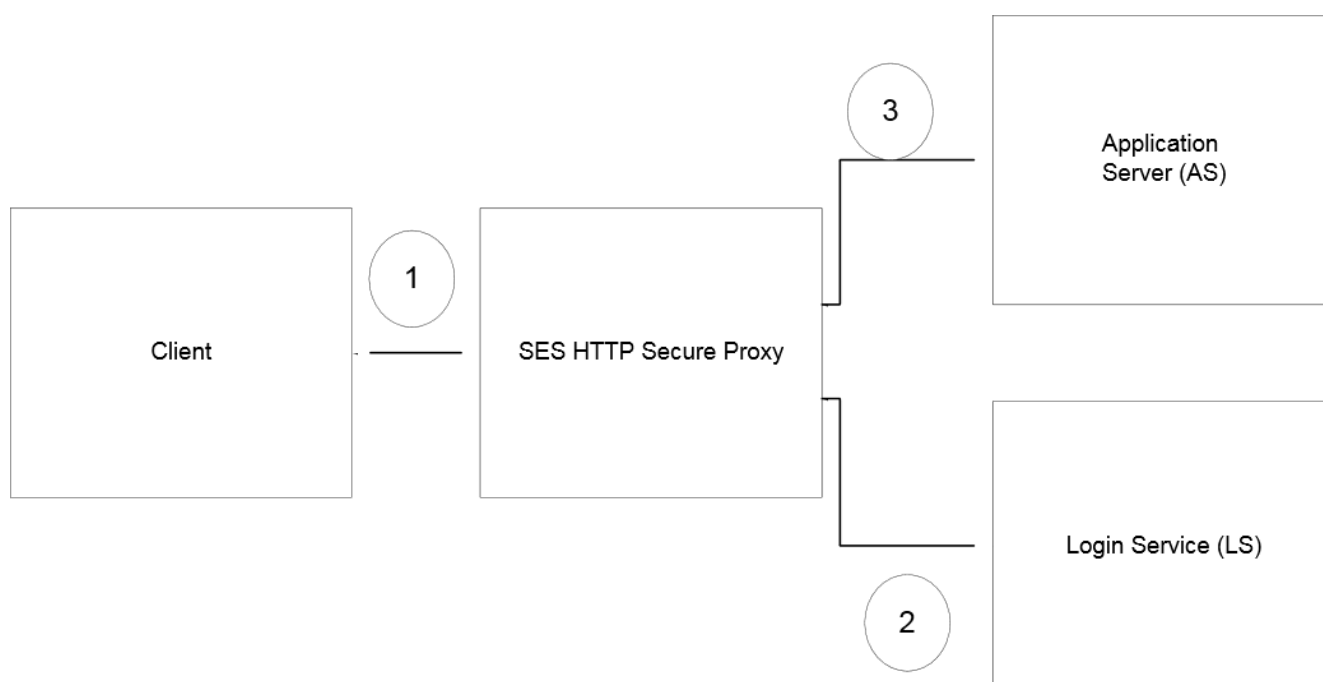


Figure 8.1: Login Service interface

8.2 Authorization Request [\[4.1 4.2 4.3 4.4 4.5\]](#)

This request is sent on the first request after denied access only and generated out of "[6.7 Denied Access Response \[3.4\]](#)".

The sentence above is not entirely correct! More precisely behaviour of HSP and SLS is as follows:



If the request contains a RequestedPage parameter as the first GET parameter, the HSP sends req az headers for the location of the RedirectPage. If there is no such RequestedPage header, the HSP determines req az headers for the AuthorizedPath location of the current location (i.e. the login location). Corresponding req az headers are then always sent (if there are any).

In SLS <4.20.0 this could cause login models to be reset in the middle due to req az headers being sent again (especially if authorizations were set for the root location "/"). Since SLS 4.20.0, req az headers are only considered if there is a RequestedPage parameter or if it is the first request of the SLS login session, and otherwise ignored.

In combination, this effectively achieves in practice the same effect as what the first sentence above stated.

Table 8.1: [Login Service Interface - Authorization Request \[4.1 4.2 4.3 4.4 4.5\]](#) - Information Items

Items		Name, Description, Conditions, Sample
AS Interface Items		See "6.5 Standard Request [7.1]"
Requested Page <option>		Item Type: Requested Page Encoding: Plain Container: HTTP Header Name: RequestedPage Sample: RequestedPage: <Requested Page>
Login Request Key <option>		Item Type: HSP Login Request Key Encoding: Plain Container: HTTP Header Name: HSPClientDataKey Sample: HSPClientDataKey: <HSP Login Request Key>
Post Data Storage Info <option>		Item Type: Post Data Storage Info Encoding: Plain Container: HTTP Header Name: PostDataStorageInfo Sample: PostDataStorageInfo: <Post Data Storage Info>
Session Info <option>		Item Type: Session Info Encoding: Plain Container: HTTP Header Name: SessionInfo
	Login Credentials	Login Credentials of different validities have different names. Common attributes are: Encoding: Plain Container: HTTP Header Item Type: Login Credentials
	Current Login Credentials <optional>	Login Credentials for the current (LS) location. Set if HSP Credential is valid and not expired . Same as AS interface. Name: LoginCredentials
	Expired Login Credentials <optional>	Login Credentials for the current (LS) location. Set if HSP Credential is expired (Inactivity-time-out only), but otherwise valid . Name: ExpiredLoginCredentials
	Login Credentials from Requested Page	Login Credentials for the Requested Page (where the Denied Access occurred). Set only if HSP Credential is valid and not expired . Name: ReqLoginCredentials



Table 8.1: (continued)

Items	Name, Description, Conditions, Sample
Expired Login Credentials from Requested Page	Login Credentials for the Requested Page (where the Denied Access occurred). Set if HSP Credential is expired (Inactivity-time-out only), but otherwise valid . Name: ReqExpiredLoginCredentials
Session Attributes	The session attributes are sent in two lists. One set contains the attributes of the session used for the current (Login Service's) URI, one for the requested page's URI. In most configurations, the sets are identical.
Session Attribute List	Session attributes from the user's session used to access the Login Service location. Item Type: <i>Session Attribute</i> Encoding: String Encoded Container: Enumerated HTTP Header Name: "HSP_AC_SESSION_ATTRIBUTES-%d" Sample: HSP_AC_SESSION_ATTRIBUTES-0: < <i>Session Attribute</i> > HSP_AC_SESSION_ATTRIBUTES-1: < <i>Session Attribute</i> > HSP_AC_SESSION_ATTRIBUTES-2: < <i>Session Attribute</i> >
Session Attribute List from Requested Page	Session Attributes from the user's session used to access the Requested Page. Item Type: <i>Session Attribute</i> Encoding: String Encoded Container: Enumerated HTTP Header Name: "HSP_AC_SESSION_ATTRIBUTES-REQ-%d" Sample: HSP_AC_SESSION_ATTRIBUTES_REQ-0: < <i>Session Attribute</i> > HSP_AC_SESSION_ATTRIBUTES_REQ-1: < <i>Session Attribute</i> > HSP_AC_SESSION_ATTRIBUTES_REQ-2: < <i>Session Attribute</i> >
Necessary access control attributes to access the requested page.	Required application authorization identifier to access the Requested Page. Item Type: <i>Session Attribute</i> Encoding: String Encoded Container: Enumerated HTTP Header Name: "HSP_AC_SESSION_ATTRIBUTES-REQAZ-%d" Sample: HSP_AC_SESSION_ATTRIBUTES_REQAZ-0: < <i>Session Attribute</i> > HSP_AC_SESSION_ATTRIBUTES_REQAZ-1: < <i>Session Attribute</i> >

8.3 Successful Login Response (Authorization) [5.1]

The authorization response authorizes the client for a specific Access Area and URI path. The access area is specified by the Login Service, whereas the URI path is configured in the HSP.

Usually, this response is sent which a HTTP "302 Moved Temporarily" status which redirects the client back to the application.

**Note**

If the SRManager is configured to use URL rewriting for external session representation, the login service must use HTTP "302 Moved Temporarily".

Table 8.2: [Login Service Interface - Successful Login Response \(Authorization\) \[5.1\]](#) - Information Items

Item	Name, Description, Conditions, Sample
Access Area Authorization	Authorized the client for the specified Access Area. Item Type: Access Area Authorization Encoding: Plain Container: HTTP Header Name: "AccessArea" Sample: AccessArea: <URL encoded Access Area Authorization >
Login Request Key <optional>	Specifies the login request key. This value must be specified in order to support POST request playback. For backward compatibility reasons, type does match container name. Item Type: HSP Login Request Key Encoding: URL encoded Container: URL parameter in the "Location" Header. Name: "HSPClientDataKey" Sample: ?HSPClientDataKey=< HSP Login Request Key >
LoginCredentials <optional>	Sets Login Credentials for this Authorization Item Type: Login Credentials Encoding: Plain Container: HTTP Header Name: LoginUserData Sample: LoginUserData: < Login Credentials >
SSL Session Id Lock Control Data	Controls SSL Session ID Lock Item Type: SSL SID Lock Control Encoding: Plain Container: HTTP Header Name "HSP_SE_LOCK_SSL_SID" Sample: HSP_SE_LOCK_SSL_SID: < SSL SID Lock Control >
Session Attribute List	Session Attributes to use for the new authorized session. Any previous session attribute within the authorized path is deleted at successful user authentication and only the new attributes are stored within the user's session. Item Type: Session Attribute Encoding: String Encoded Container: Enumerated HTTP Header Name: "HSP_AC_SESSION_ATTRIBUTES-%d" Sample: HSP_AC_SESSION_ATTRIBUTES-0: < Session Attribute > HSP_AC_SESSION_ATTRIBUTES-1: < Session Attribute > HSP_AC_SESSION_ATTRIBUTES-2: < Session Attribute >



Chapter 9

HSP Internal Interface

The table below lists the data items used inside the SES HTTP Proxy

Table 9.1: Internal data items

Name	Type	Definitions	
SessionCredentials	Cookie	Syntax	RFC 2965 Set-Cookie / Cookie value
		Description	HSP internal Cookie must not be used by AS, LS. The ES Session Credential. Note: If the SRM Session/Credential Management is configured to store this cookie (default), this cookie is not sent over the network.
		Example	
		Usage	SRM Access Control → SRM Session/Credential Management (on new Authorization by Login Service) SRM Session/Credential Management → SRM Access Control (with each request)



Chapter 10

Data Items

10.1 Item List

This chapter contains generic definitions. They are valid for all interfaces. The following table contains all data item definitions:

Table 10.1: HSP Interface - Data Items

Group, Item Type		Description
Correlation		
	Client Correlator	This identifier is used to correlate data within a Client Session: Syntax: <pre>"ClientCorrelator:" SP data</pre> <pre>data = *(<TEXT>)</pre> Sample Data: <pre>ItuSEBc2ZeJ5YbJ+1G_u4j1lBtfMDsByClvrWw\$\$</pre>
	Request Correlator	This identifier is used to correlate data within a request. Syntax: <pre>"RequestCorrelator:" SP data</pre> <pre>data = *(<TEXT>)</pre> Sample: <pre>RequestCorrelator: icfr-hsl-2002.01.11_1508 ↔</pre>
	Connection Correlator	This identifier is used to correlate requests with a client TCP connection (multiple requests over the same connection when using HTTP keep alive). Syntax: <pre>"ConnectionCorrelator:" SP data</pre> <pre>data = *(<TEXT>)</pre> Sample: <pre>ConnectionCorrelator: 8167281265628397</pre>



Table 10.1: (continued)

Group, Item Type		Description
	Application Id	This header carries a configurable application id Syntax: <pre>"ApplicationID:" SP data data = *(<TEXT>)</pre> Identifier used to name an application. Configured in the SRM. Sample: <pre>ApplicationID: Booking</pre>
	Access Control	
	Access Area Authorization	Authorizes the client for the specified access area Syntax: <pre>"Member" "Customer"</pre> Example: <pre>Member</pre>
	Logout Command	Logs the client out from a set of application (defined by Authorized path) or all applications. The corresponding sessions (AS Cookies) are deleted from the Session Store. Syntax: <pre>"Invalidate" "InvalidateAll"</pre> Example: <pre>Invalidate</pre>
	Re-authentication Command	Sets session state to re-authentication. The corresponding sessions (AS Cookies) are locked until user has successfully re-authenticated. Syntax: <pre>"<re-authentication state>"</pre> <re-authentication state> specifies the re-authentication state to initiate. LS might handle re-authentication depending on this state. All states except predefined states None, Invalid, Valid and Expired are allowed, whereas the state must consist of alphanumeric characters only. Example: <pre>CredVerify</pre>
	Session Info	Session (Login State) Information See "11.2 Session Info" Syntax: <pre>"SessionInfo:" attribute "=" value * (";" SP ↔ attribute "=" value)</pre> Sample: <pre>ReqAccessArea=Member; CredentialState=Expired</pre>
	Session Attribute	Session Attribute See "11.3 Session Attributes"
	Requested Page	Specifies the URI path which was requested by the client. Sample: <pre>/app/index.html</pre>



Table 10.1: (continued)

Group, Item Type		Description
	Post Data Storage Info	Contains information of data being stored in the SRM's POST Data Storage. Such data is a result of a POST operation done while the session's inactivity time-out was reached. Syntax: <pre>Attribute '=' value * (";" SP attribute '=' value)</pre> Attributes: <pre>"DataSize": <bytes stored> "Dropped"</pre> (More attributes may be defined in later versions.) Samples: <pre>DataSize=489 DataSize=Dropped</pre>
	HSP Login Request Key	Used to correlate denied accesses and successful login operations in the case of a inactivity time-out. This correlation is used to replay POST requests after successful re-login. Syntax: *(<TEXT>) Sample: <pre>1234556987</pre>
	Session Management	
	SCDID	Secure Data Client ID. This ID identifies each Client (Web Browser Instances) and references the Client Session which is used by the "Session/Credential Management" to store credentials represented by cookies.
	SSL SID Lock Control	Controls the SSL Session Id Lock. Currently, only enabling the SSL Session Id Lock is supported. Syntax: "On"
	Connection Attributes	
	SSL Headers	The SSL Header contain SSL Connection information such as cipher strength and certificates, see [HTTPADMIN] for details
	Client Address	Numeric IP address of the client (Web Browser). Syntax: <pre>1#3 DIGIT "." 1#3 DIGIT "." 1#3 DIGIT "." 1#3 DIGIT</pre> Sample: <pre>123.123.123.123</pre>
	HSP Listener Host	Server name and port of the Listener (HTTP or HTTPS). Syntax: <pre><host>:<port></pre> Sample: <pre>entry.tetrad.ch:443</pre> DEPRECATED: Use HSP Listener URI instead. Note Using this header conflicts with HSP architecture concepts. Use this header only when not other solution is possible. Does not contain correct port number if a port mapper is used.



Table 10.1: (continued)

Group, Item Type		Description
	HSP Listener URI	Server part of the Listener URI as seen by the client. Contains protocol specification, host and optional port. Port is supplied if client has explicitly requested a specific port. Syntax: <protocol>://<host>[:<port>] Sample: <pre>https://entry.tetrad.ch http://entry2.tetrad.ch http://entry2.tetrad.ch:8080</pre> Note Using this header conflicts with HSP architecture concepts. Use this header only when no other solution is possible.
	Content	
	Response Filter Deactivator	This header deactivates HSP filtering of response bodies. Use this header if performance is an issue, or if you need to exempt certain content from the translation rules.
	Load Information	
	Application Server Load	Reports application server load information to the HSO, which is used by the SRManager for load-balancing. Syntax: <value> Sample: 83 <value> must a integer value between 0 and \+32000 (negative values between -32000 and 0 are possible too, but not recommended). The application server is responsible that the load information is correct and truly reflects the load of the system or other lineations (e.g. licenses) which should be observed by load-balancing. Note Load distribution has to be explicitly activated by HSP configuration.
	URL Encoding	



Table 10.1: (continued)

Group, Item Type	Description
Encode URI Headers	Server can activate URL encoding for the current response. This headers overcontrol any local configuration at SRManager. • Path encoding – Header name: 'HspUE_EncPath' – Values: "on" activates the encoding of the path withn any URL "off" deactivate the encoding of the path withn any URLs – Syntax: HspUE_EncPath: "on" "off" • Query encoding – Header name: 'HspUE_EncQuery' – Values: List of query names whose values must be encoded or '*' to encode the all query values. '\$' encodes the whole query string (including the query name). – Syntax: HspUE_EncQuery: "*" "\$" <name> * (SPACE <name>) • Form input field encoding – Header name: 'HspUE_EncQuery_INPUT' – Values: List of form input node names whose values must be encoded. – Syntax: HspUE_EncQuery: <name> * (SPACE <name>)

10.2 Session Info

Session information is sent on denied access to the Login Service. It contains credential and configuration information which are relevant for the denied access. The information is sent using named attributes. The Header itself and some of its attributes are optional and only sent to the Login Service it the attributes are available.

The attributes sent with this Item have different sources:

- User credentials
Attributes used to access the requested page.
- HSP Configuration
Values taken from the SRM configuration file. This values are either taken from the Login Server location or from the location in which the requested page resists.



Table 10.2: HSP Interface - SessionInfo syntax

Attribute	Source	Description
AuthorizedArea	User Credentials	(Optional) The authorized Access Area of the Session Credentials used to access the Requested Page. values = "Member" "Customer"
AuthorizedPath	HSP Configuration	(Optional) The authorized Path of the Session Credentials used for the Requested Page.
CredentialState	User Credentials	(Optional) The authorized Path of the Session Credentials used for the Requested Page. value = "None" "Invalid" "Valid" "Expired" "..." This value might be "Valid" even if the credentials are not sufficient for the Requested Path.
ReqAuthorizedPath	HSP Configuration	The requested, Authorized Path configured for the location given with the Requested Page Header. Taken from SRM Configuration file.
LocAuthorizedPath	HSP Configuration	Actual configured value for the login location. Taken from SRM Configuration file.
ReqAccessArea	HSP Configuration	Requested Access Area The requested Access Area, value = "Member" "Customer" The requested, authorized Path configured for the location given with the RequestedPage Header. Taken from SRM Configuration file.

10.3 Session Attributes

Session attributes are stored within HSP Sessions and customize the behavior of the SRManager for authenticated clients. Session attributes are used to generate HTTP headers, supply parameters to the Adaptive Application Integrator, grant user authorizations or to define a per-user gateway configuration.

These attributes are set through special HTTP response headers by the login service on successful authentication and cleared if the user logs out. The SES also sends existing session attributes to the SLS if an authenticated user accesses the login service again.

10.3.1 When To Create Session Attribute Headers

Note that the SES accepts Session Attribute headers from the SLS only if the SLS also sets the "AccessArea" header at the same time!

10.3.2 Session Attribute Header Syntax

The login service must set a HTTP header with the following syntax in order to create an SES session attribute:

Header name:

HSP_AC_SESSION_ATTRIBUTES-<no>

Where <no> is a numeric counter, beginning with 0 and counting up. The number has no preceeding zeroes. There must be no gaps in the numbering!

Header value:

usage="", name="", path="", value="", vtype="", encoding="", timeout=""



Some of these key/value pairs are mandatory, others are optional. For each key / value pair, the following rules apply:

- The key must be an alphanumeric string.
- The value must be a string enclosed in double quotation marks. Any forms of escaping are not supported.

This is a typical example of a Session Attribute header created by the login service:

```
HSP_AC_SESSION_ATTRIBUTES-1'=' usage="prop-header", name="userid", path="/intranet ↵  
", value="jmiller", encoding="url"
```

The most important key to note is `usage`. It defines the semantical meaning of the entire header, and also what the contents of the other keys can or must be. See the following chapters for details.

10.3.2.1 Session Attribute Header Keys

A session attribute header can consist of the following keys:

Table 10.3: Keys of an SES Session Attribute Header

Key	Mandatory	Description
usage	yes	Specifies the attribute class (usage). The meaning of the header and the contents of the other keys and values all depend on the value of this key. See "11.3.2 Usage Types" for supported values and usage details.
name	yes	Specifies the attribute name. This name must be unique within the same location (path) and usage. The name can contain any alphanumeric character, spaces are not allowed.
path	yes	Specifies the attribute path. An attribute is only evaluated if its path includes (is below or equal) the current URI. An attribute is only valid within the authorized path of the current session. The path is always relative to the current virtual server. Must not have a "/" at the end.
value	yes	Specifies the attribute value. In the case of a reference attribute, the value may contain the name of the referenced value attribute. See "10.3.1 Value Type (vtype=)" .
vtype	no	Defines the value type; see "11.3.3.4 vtype" for details.
encoding	no	Defines the encoding of the value of the key "value". See "11.3.1.2 Supported Encodings" for details on supported encoding types.
timeout	no	Only used for usage="ac-app-az" See "11.3.2.4 Application Authorization Identifier (usage="ac-app-az")" for more details

10.3.2.2 Supported Encodings

The key `encoding` can have one of the following values:

Table 10.4: Supported value encodings



Table 10.4: (continued)

url:	(default) The "value" must be URL encoded. Example header part for a case where the content of the "value" key is a string "acme:3000": ...value="acme%3A3000" encoding="url"...
base64:	The "value" is a Base64 string, containing binary data.
string:	Supported for backwards compatibility only: Means that the "value" is not encoded at all (just a plain ASCII string). Example header part for a case where the content of the "value" key is a string "acme:3000": ...value="acme:3000" encoding="string"...

10.3.2.3 Supported values for the "vtype" key

The following values can be used for the "vtype" key:

Table 10.5: "vtype" value list

direct	(default) The "value" directly contains the actual value.
ref	The actual value is stored in another (referenced) attribute. The purpose of this is to save space and network bandwidth, if multiple session attributes should be created which all contain the same value. The "value" of this attribute just contains the name of the referenced attribute. The "vtype" of the referenced attribute must be "value".
value	This attribute is being referenced by another one which has "vtype" set to "ref" and "value" to the name of this attribute.

See ["11.3.3.4 vtype"](#) for an example on how to use referenced attribute values.

10.3.3 Usage Types

10.3.3.1 HTTP Header Propagation (usage="prop-header")

Session attributes of this usage type cause the SES to generate HTTP headers which are sent to application servers and the Login Service with all subsequent requests after a successful login.

If more than one HTTP Header attribute with the same name match the request's URI, a single HTTP Header is generated where only the most exact match is used.

- "name" specifies directive name
- "value" is used as the header content.

10.3.3.2 Gateway Parameter (usage="gw-param")

A set of directives in the SRManager Gateways, such as the HTTP 1.1 Gateway, can be set using Attributes of this class.

See [\[HTTPADMIN\]](#) for supported directives.

- "name" specifies the directive name,
- "value" specifies the first parameter, leave empty ("") if unused
- "param" may specify subsequent parameters.



10.3.3.3 AAI Parameter (usage="aai-param")

This type is used to set variables which are retrieved by the Adaptive Application Integrator (AAI). See [\[HTTPADMIN\]](#) for AAI rules.

- "name" specifies the variable's name,
- "value" specifies the variable's value

10.3.3.4 Application Authorization Identifier (usage="ac-app-az")

This usage type is used to set the application authorization identifier (az) variables which are used for the per application authorization functions.

Only one application authorization identifier must be set for a location (path) and the name must correspond to the definition in the SRManager. See also the RequireAz command directive in [\[HTTPADMIN\]](#).

- "name" string which specifies the application authorization identifier
- "value" = "allow"
- Optional "timeout" in seconds to specify idle timeout of this authorization

10.3.4 Examples

The following examples shows Session Attributes as they can be set by the Login Service using HTTP Headers.

10.3.4.1 Basic

```
name="TestHeader1", path="/d1", value="V1", usage="prop-header"
```

Causes the SRManager to generate a header named "TestHeader1" having value "V1" for each access to location /d1 and below.

10.3.4.2 Gateway Parameter

```
usage="gw-param", name="HGW_Host", path="/d1", value="as.foo.com:1234"s  
usage="gw-param", name="HGW_ForceHost", path="/d1/foo", value="as3.foo.com:1234"
```

If configured, the SRManager's HTTP 1.1 Gateway sets HGW_Host, HGW_ForceHost as to the defined values for the specified path and below. See [\[HTTPADMIN\]](#) for configuration.

10.3.4.3 Exact Match

```
usage="prop-header", name="TestHeader1", path="/d1", value="V1"  
usage="prop-header", name="TestHeader1", path="/d1/d2", value="V2"
```

This sample shows the "most exact match" rule applied by the SRManager.. The attributes cause the SRManager to generate a HTTP header named "TestHeader1" having value "V1" for each access to location /d1 and below not including location /d1/d2 and below. The value used for location /d1/d2 and below is "V2"



10.3.4.4 vtype

```
usage="prop-header", name="TestHeaderValue", value=="Value==",          path="/d1/ ↔  
    member",    vtype="value"  
usage="prop-header", name="TestHeader1",      value="TestHeaderValue", path="/d1/ ↔  
    member" ,   vtype="ref"  
usage="prop-header", name="TestHeader2",      value="TestHeaderValue", path="/d1/ ↔  
    member",    vtype="ref"
```

The definitions above cause the SES to generate two HTTP headers named "TestHeader1" and "TestHeader2", both using the same, shared value "Value" referenced by the value attribute with name "TestHeaderValue".

10.4 Airlock Backend API Emulator

The module `mod_airlock_api.so` do implement parital the Airlock Backend API.

10.5 Airlock Enviroment Cookies]

The prefix for the enviroment cookie names must be configured, see Admin Guid for detailed information. The following enviroment Variables are implemented:

- <Prefix>SERVER_NAME
- <Prefix>SERVER_PORT
- <Prefix>SERVER_PROTOCOL
- <Prefix>SERVER_REQUEST
- <Prefix>REMOTE_ADDR
- <Prefix>HTTPS
- <Prefix>SSL_PROTOCOL
- <Prefix>SSL_SESSION_ID
- <Prefix>SSL_CIPHER_KEYSIZE
- <Prefix>SSL_CLIENT_CERT
- <Prefix>SSL_S_DN
- <Prefix>SSL_I_DN

Airlock Control Cookie

The control cookie name must be configured, see Admin guide for detailed information. The following control commands are implemented:

- SET_CREDENTIALS: Only the first role is taken and timeout will not be set yet, the configured credential timeout will be used, see Admin guid for detailed information.
- SESSION=TERMINATE
- AUDIT_TOKEN



Appendix A

Document References

Table A.1: References

ID	Description
SESWP	USP Secure Entry Server®, White Paper
RFCHTTP1.1	IETF RFC 2616: HTTP 1.1 Specification http://www.ietf.org
RFCHTTPSTATE	IETF RFC 2965: HTTP State Management Mechanism. http://www.ietf.org
STRONGHOLD	Stronghold Web Server Administration Guide http://www.openssl.org
HTTPADMIN	HTTP Proxy Version 2.4 Administrator's Guide
RFC2246	RFC 2246, The TLS Protocol Version 1.0, January 1999 http://www.ietf.org/rfc/rfc2246.txt
SSL302	INTERNET-DRAFT draft-freier-ssl-version3-02.txt, SSL 3.0, November 18, 1996 http://wp.netscape.com/eng/ssl3/ssl-toc.html



Appendix B

Glossary

Applet

A Java application which, in contrast to a servlet, executes on the client.

Authenticate

Verification of personal details, usually involving a username and password. Security certificates, security chipcards or fingerprint scanners are other common options for verifying an individual's credentials.

Client (computer)

A computer that is served by a server. A computer belonging to one of your customers, for example.

Cookie

A technique developed by Netscape for temporary storage of small quantities of data on a client computer and subsequently recommended as an RFC for all HTTP clients. The HTTP Proxy uses cookies in order to recognize a previously logged-in user. The content is stored in encrypted form, although cookie technology also permits unencrypted data storage.

Demon

A background process on UNIX systems, for example a web server, printer driver, file services, etc.

Header fields

Data fields transmitted in the HTTP protocol header. They allow communication between the client program (browser) and the web server. Cookies, error messages and version numbers are transmitted in the HTTP header.

HTTP

Hyper **T**ext **T**ransfer **P**rotocol

An application layer Internet protocol governing the exchange of HTML documents. HTTP is standardized in RFCs. Version 1.1 is presently widespread.

HTTPS

Secure **H**yper **T**ext **T**ransfer **P**rotocol

HTTP secured by an encrypted connection. Based on SSL.

HTTP Proxy

A component of the that provides centralized encryption, security and authentication to HTTP applications.

Redirect

Re-routing of one URL to another. Usually involves an HTTP status 302 in the case of the HTTP Proxy.

RFC



R equest **F** or **C** omments

Standardization and regulations for Internet protocol deployment.

Servlet

An application that executes on the server - as opposed to applets, which execute on the client.

**

+ Name of a middleware product from tetrade

Session

A "conversation" between client and server: for example, while a customer fetches his data from your server, a session exists between your server and the customer's client computer.

SSL

S ecur**e** **S** ocket **L** ayer

A protocol developed by Netscape to facilitate encrypted communications for services like HTTP, FTP, SMTP, etc. Initial key exchange and the session itself use the RSA and RC4 algorithms respectively.

URL

U niform **R** esource **L** ocator

A canonical address and path specification for accessing data stored on a web server.

virtual

Unreal, not present in reality, yet having sufficient appearance of reality to fool the eye or other senses.

In the context of the HTTP Proxy, virtual paths are paths that bear no relation to the physical server layout. We can thus conceal physical interrelationships for better protection against attack.